

Small to large merging

Motivação

- Juntar conjuntos de forma eficiente, dois a dois.
- Exemplo de aplicação: union-find, kruskal

Kruskal

- Considere o algoritmo de Kruskal:
- Algoritmo de Kruskal
 - Seja G um grafo conexo e v um vértice qualquer de $V(G)$.
 - $T \leftarrow$ um grafo vazio
 - Enquanto $|E(T)| < |V(G)|-1$ faça
 - $e \leftarrow$ uma aresta $e=(u,v)$ de G sendo que e é uma aresta de custo mínimo que não gera um ciclo em T .
 - $T \leftarrow T + e + v$
 - Retornar T
- Qual a dificuldade na implementação deste algoritmo?

Kruskal

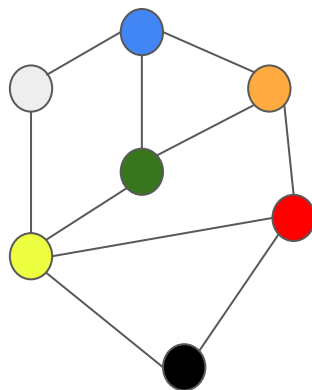
- Detectar ciclos no algoritmo de Kruskal:
 - Simples e eficiente com union-find.
 - Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?

Kruskal

- Detectar ciclos no algoritmo de Kruskal:
 - Simples e eficiente com union-find.
 - Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$

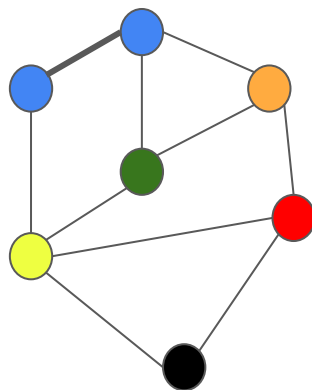
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Exemplo de pior caso:



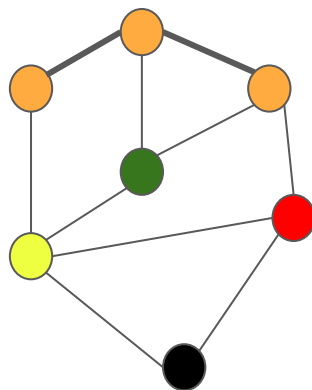
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Exemplo de pior caso:
- Operações: 1 (cinza->azul) +



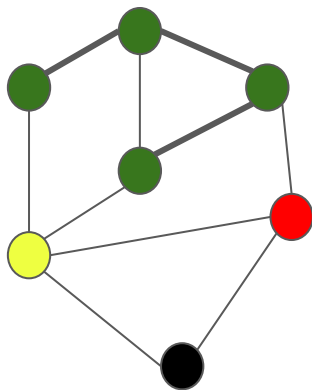
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Exemplo de pior caso:
- Operações: 1 (cinza->azul) + 2 (azul -> amarelo)



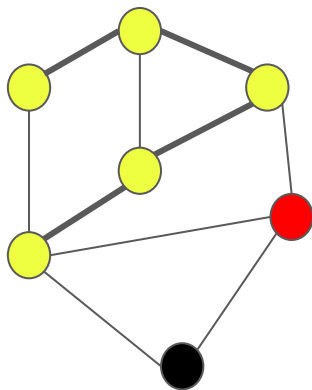
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Exemplo de pior caso:
- Operações: 1 (cinza->azul) + 2 (azul -> amarelo) + 3 (amarelo -> verde)



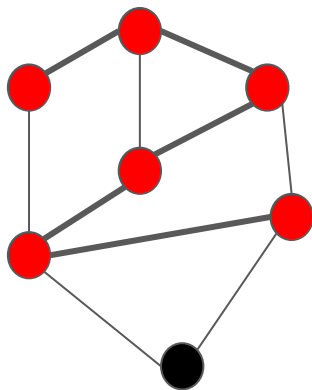
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Exemplo de pior caso:
- Operações: 1 (cinza->azul) + 2 (azul -> amarelo) + 3 (amarelo -> verde) + 4 (verde -> amarelo claro)



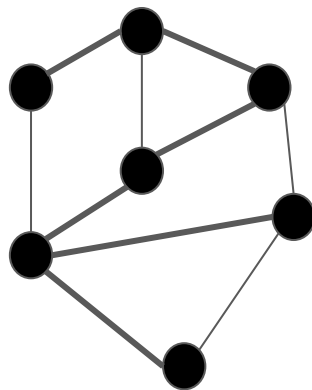
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Exemplo de pior caso:
- Operações: 1 (cinza->azul) + 2 (azul -> amarelo) + 3 (amarelo -> verde) + 4 (verde -> amarelo claro) + 5 (amarelo -> vermelho)



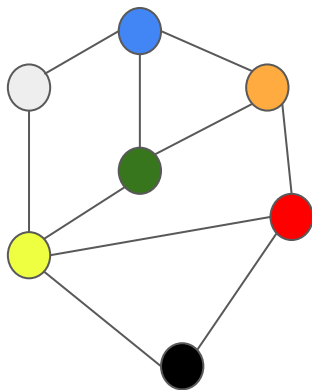
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Exemplo de pior caso:
- Operações: 1 (cinza->azul) + 2 (azul -> amarelo) + 3 (amarelo -> verde) + 4 (verde -> amarelo claro) + 5 (amarelo -> vermelho) + 6 (vermelho -> preto)



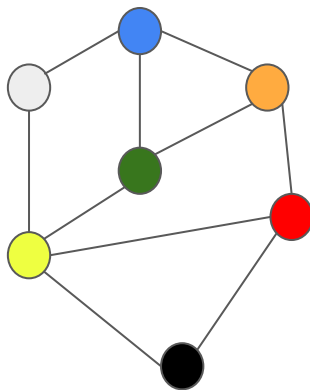
Kruskal

- Qual o custo se (em vez de utilizar uma union-find) utilizar uma DFS/BFS para marcar os CCs com códigos?
 - $O(n^2)$, pois cada BFS tem custo $O(n)$
- Veja o código `kruskalSemSmallToLarge.cpp` e teste-o com a entrada [exemploKruskal2.in](#) (não é exatamente o pior caso)
- Alguma ideia sobre como fazer melhor?



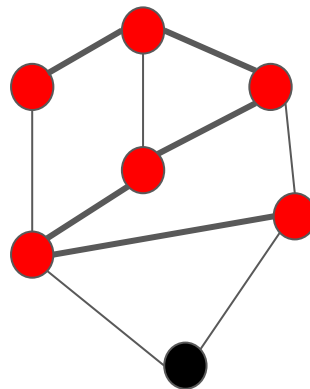
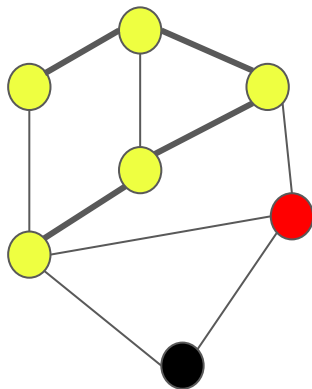
Kruskal

- Alguma ideia sobre como fazer melhor?
 - E se sempre marcarmos o menor CC com a cor do maior?
 - Ou seja, vamos SEMPRE unir o menor conjunto no maior
 - Adapte o código do Kruskal para funcionar assim e teste novamente! (basta adicionar um if com swap...)



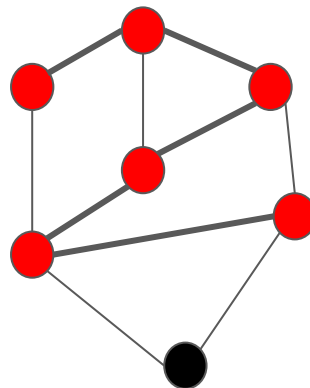
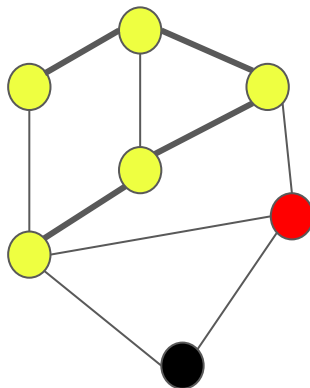
Kruskal

- Por que funciona?
 - Antes, poderíamos fazer N uniões com complexidade $1, 2, 3, \dots, N \rightarrow O(N^2)$



Kruskal

- Por que funciona?
 - Antes, poderíamos fazer N uniões com complexidade $1, 2, 3, \dots, N \rightarrow O(N^2)$
 - Agora, ao unir um conjunto grande com um pequeno a complexidade é sempre baixa
 - Unir conjuntos de tamanho parecido continua com o mesmo tempo, mas isso dobra o tamanho dos conjuntos $\rightarrow$ ocorre poucas vezes.
 - No geral, small to large merging fica com complexidade $O(N \log N)$ ou $O(N \log^2 N)$



Small to large merging

- Small to large
 - Kruskal foi apenas um exemplo. (obviamente, é melhor usar uma union find tradicional...)
 - Praticar exemplos para fixar a ideia!
 - Em geral, resolvemos os problemas de forma trivial (exemplo: kruskal sem Union Find) e, depois, apenas colocamos um swap para fazer o small to large.

